

Active technology watch

<https://github.com/aiekick/ImGuiFileDialog>

https://www.youtube.com/watch?v=VLZjd_Y1gJ8&t=195s

A potential concurrent of C++ : <https://inductive.no/jai/>

<https://www.youtube.com/watch?v=klE9bzWJbHc>

<https://www.youtube.com/watch?v=YspUkfTY0dY>

Other concurrent :

<https://odin-lang.org/>

<https://jangafx.com/software/embergen>

Graphical :

<https://contribute.qt-project.org/>

<https://github.com/ocornut/imgui>

Profilers :

<https://github.com/wolfpld/tracy>

<https://github.com/bombomby/optick>

<https://github.com/Celtoys/Remotery>

Doxygen :

<https://www.mkdocs.org/>

Set up a regular watch on topics linked to the project: languages, frameworks, protocols, tools, standards, architectures, etc.

Done

https://docs.vulkan.org/tutorial/latest/Building_a_Simple_Engine/Engine_Architecture/01_introduction.html

<https://gpuopen.com/>

<https://developer.nvidia.com/blog/>

<https://learnopengl.com/>

<https://www.flecs.dev/flecs/>

<https://github.com/skypjack/entt>

<https://bitsquid.blogspot.com/>

<https://gameprogrammingpatterns.com/>

<https://www.youtube.com/@CppCon>

<https://www.youtube.com/@cppweekly>

<https://isocpp.org/blog>

<https://github.com/wolfpld/tracy>

<https://box2d.org/documentation/>
<https://jrouwe.github.io/JoltPhysics/>
<https://gafferongames.com/>
<https://cmake.org/documentation/>
<https://learn.microsoft.com/en-us/vcpkg/>
<https://docs.github.com/en/actions>
<https://www.reddit.com/r/gameenginedevs/>
<https://www.reddit.com/r/vulkan/>
<https://www.youtube.com/@TheCherno>
<https://www.youtube.com/@javidx9>
<https://arxiv.org/search/?query=game+engine+architecture&searchtype=all>

Use varied, credible sources: technical blogs, scientific publications, official documentation, GitHub repos, newsletters, Discord, X, etc.

Done (once the things at the top have been visited and understood, that will be OK)

Identify emerging trends/solutions that are useful for the project (not merely curiosity).

Done

Documentation & analysis

Write comparative briefs or analysis notes on the explored tech: advantages, drawbacks, learning curve, project fit.

Done for the profiler : [Comparative study](#)

Done : [Comparative study](#)

Produce one or more technical benchmarks between approaches/solutions (performance, scalability, maintainability, ecosystem, etc.).

WIP

Present a reasoned decision on choices made (or rejected).

Done in comparative's study

Concrete application

Carry out 1–2 concrete experiments (POC, mini-project, tested integration in the repo)

Done for the profiler : [Optick](#) | [Remotery](#) | [Tracy](#)

Needs to do it for another thing (optional)

Integrate at least one new technology into the project with working proof and justification.

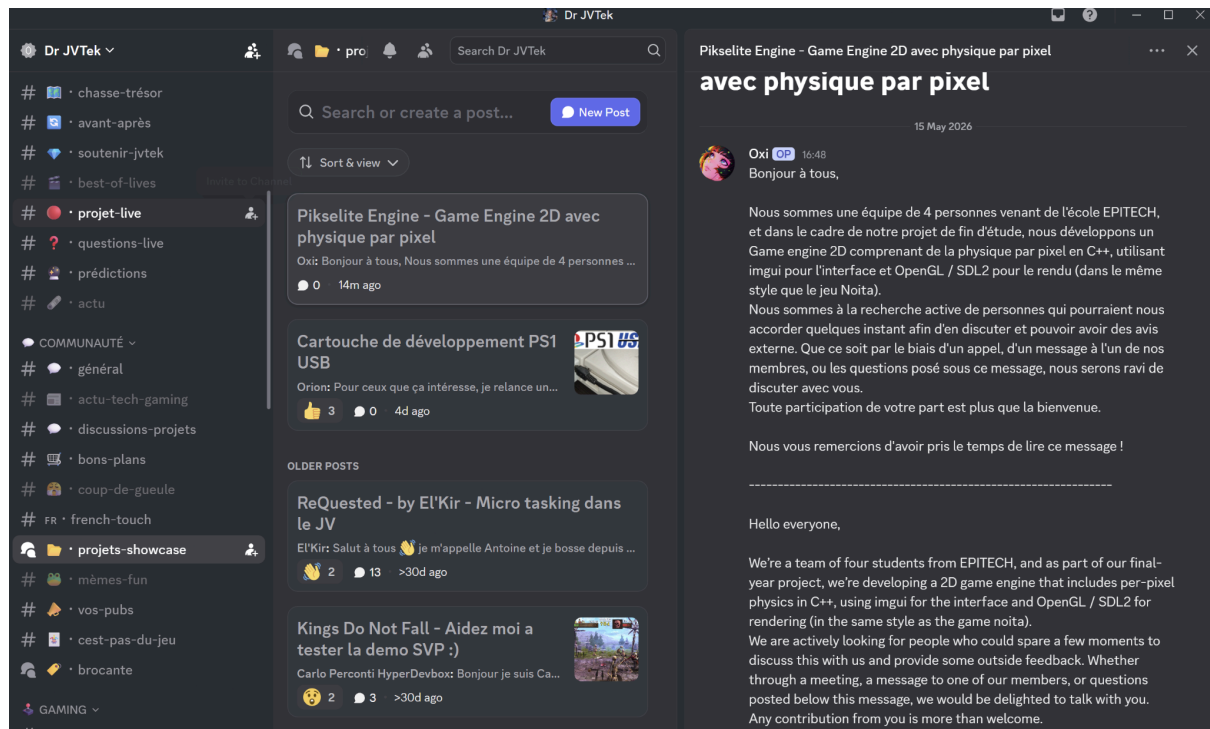
Done with the profiler

Update the project (docs, architecture, code) to reflect adopted evolutions.

WIP

Sharing & openness

Participate actively in tech communities: exchanges on Discord, StackOverflow, GitHub, specialized forums, etc.



Document your advances (README, changelog, wiki, GitHub issues), and share learnings within the team.

Github Issues, and the changelog is coming

Clear, justified architecture

Present the project's architecture: components, services, dependencies, data flows, deployment, versioning, etc.

[Archi doc](#)

Justify structural choices: selected technologies, code organization, databases, APIs, frameworks, etc.

Done with the [comparative study](#).

Complete technical documentation

Provide a structured README

[README](#) (Done)

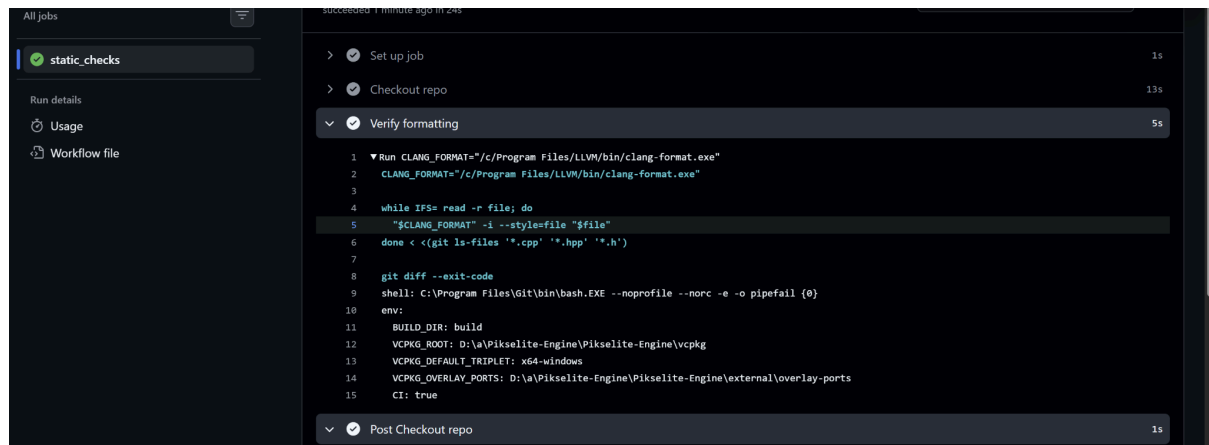
Create at least one advanced document: architecture diagram, technical use case, business logic, technical roadmap, etc.

[Archi mindmap](#)

Quality, reliability & security

Enforce code standards (linting, conventions, naming, typing).

Code formatter (clang-format) in the CI + CppCheck



Run quality analysis tools (SonarQube, ESLint, Pylint, etc.).

CppCheck

Provide unit or validation tests on at least one critical module.

Unit test done on the core of the game engine : [tests](#)

Implement initial error handling, logging, and minimal security (auth, sanitization, backups, ...).

throw but not catching all, and the available tools are tl::expected, boost.outcome, spdlog
WIP

Identify & approach experts

Clearly define the technical need or topic to challenge: architecture choice, framework, security, performance, scalability, AI, etc.

Done - [mémo](#)

1. Présentation du projet

Nous développons ~~Pikselite~~ Engine, un moteur de jeu 2D avec une interface inspirée de Unity, basé sur une architecture ECS. Le projet est réalisé par une équipe de 4 développeurs. (Montrer [Noita](#))

2. État actuel du projet

- Mise en place d'une première architecture ECS
- Début d'implémentation des systèmes de base
- Premiers tests de simulation pixel (prototype)
- Réflexion en cours sur :
 - la structure des données pour la simulation

3. Démonstration

- Simulation actuelle
- Organisation ECS
- État de l'outil

4. Objectifs de l'entretien

- Valider nos choix d'architecture
- Identifier les risques techniques majeurs
- Obtenir des recommandations concrètes pour la suite du développement

5. Questions

Physique & Simulation

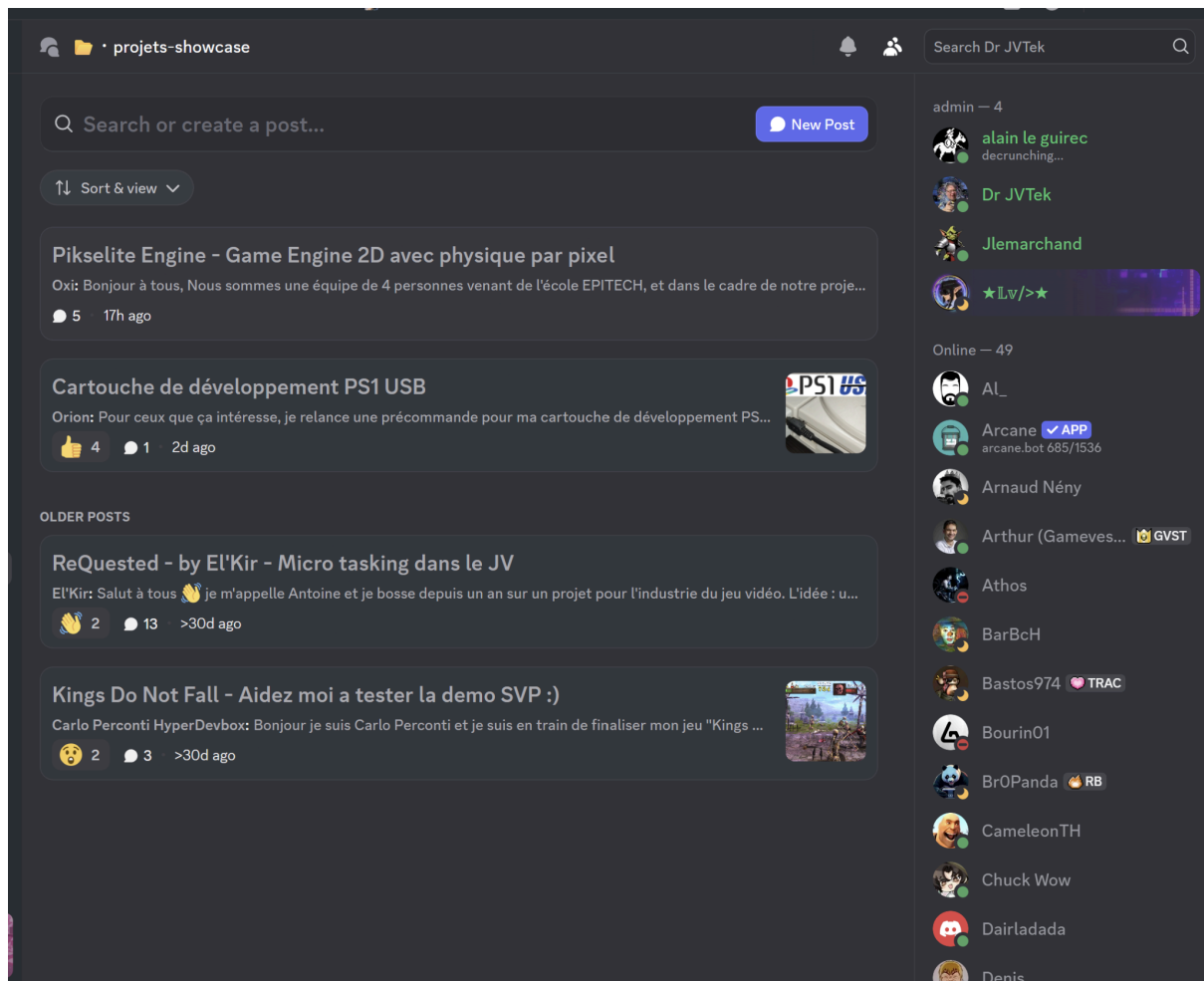
- Pour une simulation type Noita, recommanderais-tu une représentation en grille dense, chunk ou sparse ? (multi thread ?)

Architecture & Performance

- Quels sont les signaux concrets (notamment en profiling) qui indiquent un problème d'architecture plutôt qu'un simple manque d'optimisation ?
- Y a-t-il des métriques ou patterns que tu surveilles en priorité ?

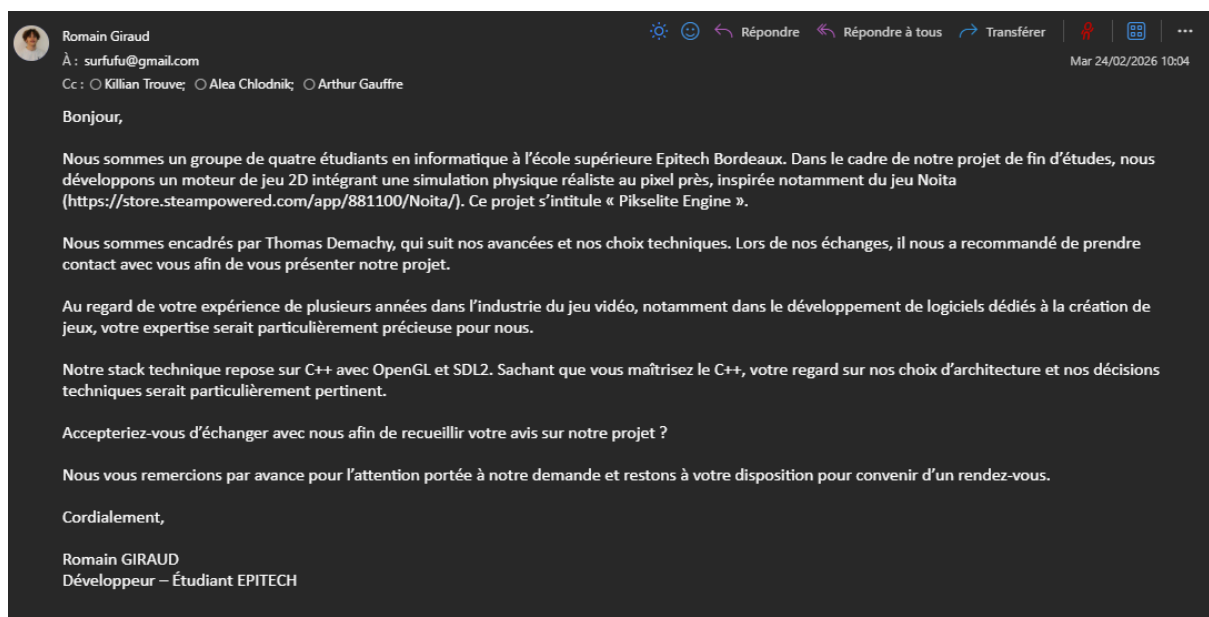
Proactively search for relevant experts: GitHub contributors, LinkedIn engineers, community leads, teachers, CTOs, alumni, etc.

Done - Sur le discord : [Arnaud Nény](#) | [Alain le guirec](#) et biens plus



Professional outreach: clear message, precise question, appropriate channel.

Done





Oxi OP 15/05/2026 16:48



Bonjour à tous,

Nous sommes une équipe de 4 personnes venant de l'école EPITECH, et dans le cadre de notre projet de fin d'étude, nous développons un Game engine 2D comprenant de la physique par pixel en C++, utilisant imgui pour l'interface et OpenGL / SDL2 pour le rendu (dans le même style que le jeu Noita).

Nous sommes à la recherche active de personnes qui pourraient nous accorder quelques instant afin d'en discuter et pouvoir avoir des avis externe. Que ce soit par le biais d'un appel, d'un message à l'un de nos membres, ou les questions posé sous ce message, nous serons ravi de discuter avec vous.

Toute participation de votre part est plus que la bienvenue.

Nous vous remercions d'avoir pris le temps de lire ce message !

Hello everyone,

We're a team of four students from EPITECH, and as part of our final-year project, we're developing a 2D game engine that includes per-pixel physics in C++, using imgui for the interface and OpenGL / SDL2 for rendering (in the same style as the game noita).

We are actively looking for people who could spare a few moments to discuss this with us and provide some outside feedback. Whether through a meeting, a message to one of our members, or questions posted below this message, we would be delighted to talk with you. Any contribution from you is more than welcome.

Thank you for taking the time to read this message !

Structured collaboration

Run a documented exchange: meeting, written feedback, Discord/GitHub thread, code review, etc.

Done : [meeting Nicolas Perret](#) | [meeting Philip Belhassen](#)

Optionally organize a review session with an external expert (pair programming, code review, light audit).

As it's optional it will not be done (due to the lack of time).

Implement concrete actions from the exchange: structure adjustment, technical plan change, new resources, tool adoption, process improvement.

Done, [Philip Belhassen](#) advices :

Focus on in-game physics && Check which pixels are moving and which are not

Capitalize on the exchange

Write a summary/memo: discussed points, advice received, decisions taken.

Done : [meeting Nicolas Perret](#) | [meeting Philip Belhassen](#)

Add the collaboration to the roadmap or documentation.

Done in the main [readme](#)

Valorize when possible: testimonial, mention, quote, public thanks.

Done in the main [readme](#)

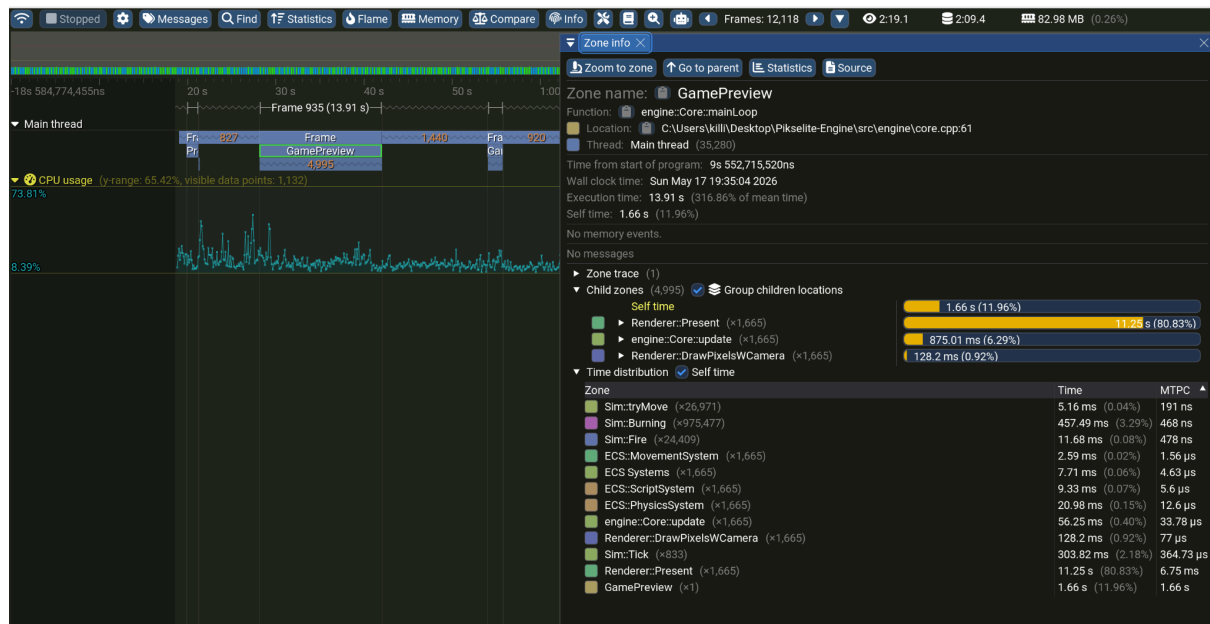
Define key technical metrics

Choose 2–3 performance indicators relevant to the project (latency, response time, memory usage, load time, bundle size, CPU, API calls, etc.).

Done : pixel simulation per-element cost, render pipeline (CPU / GPU) and ECS per-system CPU cost

Integrate these metrics into the dev cycle (dashboard, logs, third-party tools, tests, etc.).

Done with Tracy



Set up automated/manual tests

Run load tests, stress tests, and/or simulation scenarios

Done (can do more, but it's implemented) for a total of 97 tests and 64% coverage

```
38 + TEST(EcsStressTests, ComponentTypeAtExactLimit)
39 + {
40 +     engine::ComponentManager cm;
41 +
42 +     for (std::size_t i = 0; i < ecs::MAX_COMPONENT_TYPES; ++i)
43 +     {
44 +         switch (i)
45 +         {
46 +             case 0:
47 +                 cm.registerComponent<StressComp<0>>();
48 +                 break;
49 +             case 1:
50 +                 cm.registerComponent<StressComp<1>>();
51 +                 break;
52 +             case 2:
53 +                 cm.registerComponent<StressComp<2>>();
54 +                 break;
55 +             case 3:
56 +                 cm.registerComponent<StressComp<3>>();
57 +                 break;
58 +             case 4:
59 +                 cm.registerComponent<StressComp<4>>();
60 +                 break;
61 +             case 5:
62 +                 cm.registerComponent<StressComp<5>>();
63 +                 break;
64 +             case 6:
65 +                 cm.registerComponent<StressComp<6>>();
66 +                 break;
```

Run resilience tests: error handling, interruptions, unexpected behavior.

DONE (can do more, but it's implemented) for a total of 97 tests and 64% coverage

```

30 + TEST(SimulationResilienceTests, SingleSandFallsDown)
31 + {
32 +     ChunkGrid grid;
33 +     Simulation sim(grid);
34 +
35 +     // Sand at y=1 falls to y=0 (bottom)
36 +     placePixel(grid, 0, 1, Element::SAND);
37 +     sim.update();
38 +
39 +     EXPECT_EQ(grid.getPixel(0, 1).type, Element::EMPTY);
40 +     EXPECT_EQ(grid.getPixel(0, 0).type, Element::SAND);
41 + }
42 +
43 + TEST(SimulationResilienceTests, SingleWaterFallsDown)
44 + {
45 +     ChunkGrid grid;
46 +     Simulation sim(grid);
47 +
48 +     // Water at y=3 falls to y=2, then eventually to y=0
49 +     placePixel(grid, 3, 3, Element::WATER);
50 +     sim.update();
51 +
52 +     EXPECT_EQ(grid.getPixel(3, 3).type, Element::EMPTY);
53 +     EXPECT_EQ(grid.getPixel(3, 2).type, Element::WATER);
54 + }

```

Run comparative efficiency tests: optimized vs. initial version.

WIP

Implement optimizations

Analyze bottlenecks and friction points.

Done in [analysis.md](#)

Current Analysis — Pikselite-Engine

Bottlenecks

1. Pixel Simulation — the #1 hot path

`element.cpp:383-433` runs every fixed timestep (60 Hz). For each frame:

- `resetUpdatedFlags()` (line 468) iterates **all pixels in all chunks**, even empty ones
- `orderChunksForUpdate()` (line 478) copies all chunk entries into a vector then `std::sort`s them — heap allocation + $O(n \log n)$ every tick
- Each non-empty pixel dispatches through a function pointer (`def.update`), preventing inlining
- `rand()` is called per-pixel for dispersion direction and fire spread — not thread-safe, high contention

At $200 \times 200 = 40K$ pixels, this is 40K+ iterations $\times$ several passes per frame.

2. Marching Squares + Triangulation — all-or-nothing rebuild

`detectRegions()` (`element.cpp:539-629`) runs whenever `regionsDirty`:

- Destroys **all** existing Box2D bodies before rebuilding
- Flood fill $\rightarrow$ marching squares $\rightarrow$ RDP simplification $\rightarrow$ ear-clipping triangulation
- Ear-clipping (`triangulateRegion` , line 1056) has an $O(n^2)$ worst-case with a hard guard of 10K iterations
- `visitedForRegions` `unordered_set` is rebuilt from scratch each time
- No incremental update — one pixel change triggers a full region rebuild

3. `buildRenderPixels()` — redundant pixel marshaling

`core.cpp:495-558` converts the entire simulation grid $\rightarrow$ `std::vector<graphics::Pixel>` every frame. Each pixel does:

- `(cx * 32 + x) * PIXEL_SIZE` float math
- Index into `colorPalette` with float division by 255.0
- If burning: two color lookups + `glm::mix` + `glm::clamp`
- `push_back` into a vector (reallocations: `reserve(10000)` is a guess, not based on actual count)

This is done **unconditionally**, even if nothing changed.

4. Double sprite iteration

`drawSpritesBelowLayer` + `drawSpritesAboveLayer` (`core.cpp:891-1021`) are ~95% duplicate code that both:

- Linear scan all entities (5000 max)
- Filter, sort by layer, then render
- No spatial culling, no caching

5. Lua $\rightarrow$ C++ double marshaling

`ScriptSystem::update()` (`scriptSystem.hpp:108-154`) per-entity per-frame:

- `buildEntityTable()` copies C++ component data into a new Lua table
- Lua `update(dt)` runs user code that modifies the table
- `applyEntityTableChanges()` reads the table back into C++ components
- This round-trips all component fields even if only one changed

6. Build pipeline I/O

`core.cpp:560-747` uses `_popen` with tiny 128-byte buffers, reconfigures CMake from scratch for every build, and blocks a thread on synchronous process I/O.

Modify one or more technical bricks (queries, processing, cache, code, ...).

WIP | Done (colliders)

Made the built game more performant than the preview.

Justify trade-offs (quality vs. performance vs. cost vs. time).
Done in [analysis.md](#)

Trade-off Justifications

Quality vs. Performance

Trade-off	Choice	Rationale
Region detection	Full rebuild vs. incremental	Full rebuild is simpler to implement correctly; for the target game scale (small pixel worlds), the $O(n^2)$ ear-clip is tolerable
Pixel grid	<code>unordered_map</code> chunks vs. dense array	Sparse worlds benefit from chunk-based storage; hash lookups per pixel are the cost of not needing a fixed world size
Collision pipeline	MS → RDP → ear-clip in one pass	Off-the-shelf algorithms, no external dependency (no poly2tri, no CGAL); easy to debug
<code>rand()</code> vs. <code><random></code>	<code>rand()</code>	Simple, deterministic per-seed; adequate for pixel simulation randomness; avoids complexity of proper RNG seeding/threading
Fixed timestep cap	0.25s accumulator clamp	Prevents spiral-of-death; simple to understand; trade-off: physics skips if frame rate tanks

Performance vs. Cost (Development Complexity)

Trade-off	Choice	Rationale
ScriptSystem header-only	No .cpp split	Avoids managing a separate compilation unit, forward declarations, and linker symbols for what is essentially a single-class system
<code>std::any</code> components	Type-erased storage	Flexible component registration without code generation; no macros/templates for component registration
<code>buildRenderPixels</code> per frame	No dirty tracking	Keeping a clean/dirty bitmap adds complexity; for small grids the full rebuild is fast enough
<code>_popen</code> for build	No proper process library	Cross-platform process spawning is complex; <code>_popen</code> works on Windows and keeps the build pipeline ~50 lines
No spatial index for sprites	Linear scan + sort	For <1000 sprites the $O(n \log n)$ sort dominates; a spatial hash would add ~200 lines with marginal gain at target scale

Time vs. Quality

Trade-off	Choice	Rationale
Tracy profiler always present	<code>#error</code> if disabled	Forces profiling on in debug; the developer values observability over compile-time flexibility
Test suite scope	18 test files with load/stress/resilience	High investment in correctness for the simulation and ECS — the two most complex subsystems; fewer tests for UI/editors
No incremental compilation	Full CMake reconfigure per build	Build pipeline is a late addition; the <code>_popen + cmake --build</code> pattern works but doesn't use CMake's server mode or presets
Commented-out tests	Abandoned rather than deleted	Preserves history of what was intended; risk: they won't compile if the API changes

Cost (Real-world Resource Trade-offs)

Resource	Observation
vcpkg binary cache	CI uses <code>VCPKG_BINARY_SOURCES</code> with NuGet — avoids recompiling dependencies
Self-hosted Windows runners	Free CI minutes; but Windows-only, no macOS/Linux coverage
Header-only libraries	sol2, glm, nlohmann-json, stb — zero build cost for dependencies; compile-time cost shifts to the engine's TUs
Single monolithic editor exe	No DLL/shared library split; simpler deployment, faster link, but larger binary
Tracy	Free profiler with high value; the <code>TracyClient.dll</code> copy step is a minor CI cost

Summary: The engine's bottlenecks are in the pixel simulation loop (per-pixel function pointer dispatch, full-grid flag reset, chunk sorting) and the collision rebuild pipeline (all-or-nothing). The biggest friction points are the 1133-line header-only ScriptSystem (compile time) and the `rand()` usage (prevents parallel sim). The trade-offs consistently favor implementation simplicity and debuggability over peak performance — reasonable for a 2D pixel engine targeting small-to-medium game worlds.